

QIE Pass

API Reference

Identity Verification Integration Guide

Overview

The QIE Pass API enables your platform to verify users using their QIE identity credentials. Once a user has completed KYC through the QIE Wallet, their verified credentials can be requested and claimed by any integrated partner, without requiring the user to re-submit documents.

QIE Pass supports **two integration paths**. You send **exactly one** identity field on each request:

Path	Identity field	When to use
Option 1 — Wallet	<code>identifier</code>	The user already has a QIE Wallet (verified or not). Default and recommended path.
Option 2 — Web NEW	<code>partnerUserRef</code>	The user is brand new to QIE with no wallet. QIE hosts onboarding (wallet creation, KYC, consent) on a redirect page.

Both paths converge on the same poll-and-claim steps. Integration follows four sequential steps:

1	Create a verification request for your user
2	Handle the user's KYC status (already verified, or needs to complete KYC first)
3	Poll the request until the user approves or rejects
4	Claim the cryptographically-signed verified data

Authentication

Every API request must include three signed headers. These headers authenticate your request and protect against replay attacks.

Header	Value
<code>X-Public-Key</code>	Your public API key (begins with <code>pk_test_...</code>)
<code>X-Signature</code>	HMAC-SHA256 signature — see formula below
<code>X-Timestamp</code>	Current Unix timestamp in milliseconds

Signature Formula

Construct the signature by concatenating your public key and the current timestamp, then sign with your API secret using HMAC-SHA256:

Signature Formula

```
message    = publicKey + timestamp
signature  = HMAC-SHA256(message, apiSecret)
```

Code Examples**Node.js**

```
const crypto = require('crypto');

function getAuthHeaders(publicKey, apiSecret) {
  const timestamp = Date.now().toString();
  const message   = publicKey + timestamp;
  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(message)
    .digest('hex');
  return {
    'X-Public-Key': publicKey,
    'X-Signature': signature,
    'X-Timestamp': timestamp,
    'Content-Type': 'application/json',
  };
}
```

Python

```
import hmac, hashlib, time

def get_auth_headers(public_key, api_secret):
    timestamp = str(int(time.time() * 1000))
    message   = public_key + timestamp
    signature = hmac.new(
        api_secret.encode(), message.encode(), hashlib.sha256
    ).hexdigest()
    return {
        'X-Public-Key': public_key,
        'X-Signature': signature,
        'X-Timestamp': timestamp,
        'Content-Type': 'application/json',
    }
```

Important

The timestamp must be within 5 minutes of server time, requests outside this window will be rejected. Generate a fresh signature for every request. Signatures cannot be reused.

Configuring Requested Claims

Before making verification API calls, decide which identity claims your platform needs. This is a one-time configuration decision, the same claim set is used for all users on your platform.

Step 1 — Discover Available Claims

GET /partners/verification-requests/available-claims

Returns the full list of claims QIE can provide

Step 2 — Choose What You Need

Request only the claims your use case genuinely requires. Users see exactly which fields you are asking for before they approve. Fewer claims leads to higher approval rates.

Example – Minimal claim set

```
{
  "requestedClaims": ["age_over_18", "citizenship"]
}
```

Available Claims Reference

Personal Information

Claim	Description
firstName	Legal first name
lastName	Legal last name
dateOfBirth	Date of birth (YYYY-MM-DD)
citizenship	Country of citizenship (ISO 3166-1 alpha-2)
nationality	Nationality (ISO 3166-1 alpha-2)
gender	Gender

Age Verification (boolean — true/false)

Claim	Description
age_over_18	User is 18 or older
age_over_21	User is 21 or older
age_over_25	User is 25 or older
age_over_65	User is 65 or older
age_18_to_25	User is between 18 and 25
age_26_to_65	User is between 26 and 65

Citizenship (boolean — true/false)

Claim	Description
is_us_citizen	User is a US citizen
is_eu_citizen	User is an EU citizen
is_us_national	User is a US national
is_eu_national	User is an EU national

Address

Claim	Description
address.country	Country of residence
address.city	City
address.street	Street address
address.postalCode	Postal / ZIP code
address.state	State or region

ID Document

Claim	Description
<code>idDocument.type</code>	Document type (passport, national ID, etc.)
<code>idDocument.country</code>	Issuing country
<code>idDocument.issuedDate</code>	Date issued
<code>idDocument.validUntil</code>	Expiry date

Step 1 — Create a Verification Request

POST /partners/verification-requests

Initiates a QIE Pass verification request for a specific user

Send **exactly one** of `identifier` or `partnerUserRef` — never both, never neither.

Option 1 — identifier (user has a QIE Wallet)

Request Body

Field	Type	Description
<code>identifier</code>	string	The user's QIE identity — wallet address, QIE Pass ID, or QIE domain (see formats below)
<code>requestedClaims</code>	string[]	Array of claim keys to request from the user (configured once per integration)

Identifier Formats

Format	Example
Primary EVM wallet address	<code>0x1a2b3c4d...</code>
QIE Pass ID	<code>@username</code>
QIE Domain	<code>name.qie</code>

Request – POST /partners/verification-requests

```
{
  "identifier":      "0x1a2b3c4d...",
  "requestedClaims": ["age_over_18", "citizenship"]
}
```

Response (200 OK)

```
{
  "success": true,
  "data": {
    "requestId": "pvr_1744531200000_9ezjx8999",
    "status": "pending_consent",
    "walletAddress": "0x1a2b3c...",
    "requestedClaims": ["age_over_18", "citizenship"],
    "expiresAt": "2026-04-13T11:00:00.000Z",
    "userStatus": "verified",
    "createdAt": "2026-04-13T10:00:00.000Z"
  }
}
```

Response Fields

Field	Type	Description
requestId	string	Unique request ID — store this for polling and claiming
status	string	<code>pending_consent</code> (user verified) or <code>pending_kyc</code> (user not yet verified)
walletAddress	string	Resolved wallet address linked to the identifier provided
requestedClaims	string[]	The claims that were requested
expiresAt	string	ISO 8601 timestamp — request expires at this time
userStatus	string	<code>verified</code> or <code>not_verified</code>
redirectUrl	string	Only present when status is <code>pending_kyc</code> — redirect the user here to complete KYC
createdAt	string	ISO 8601 timestamp of request creation

Idempotency

If a pending request already exists for this user (`pending_kyc` or `pending_consent`) within your organisation, the existing request is returned — a new one is not created.

The `requestId` and `expiresAt` will reflect the original request. The user will receive a fresh push notification regardless.

Option 2 — partnerUserRef (new user, no QIE Wallet) NEW

For users completely new to QIE with no wallet yet. Send your own stable reference instead of an identifier, plus a `returnUrl`. QIE hosts the entire onboarding — wallet creation, KYC, and consent — on a redirect page, then returns the user to your `returnUrl`.

Request Body

Field	Type	Description
partnerUserRef	string	Your own stable internal user ID. Used for correlation and deduplication
requestedClaims	string[]	Array of claim keys to request from the user
returnUrl	string	Where QIE returns the user after the hosted onboarding completes

Request – POST /partners/verification-requests

```
{
  "partnerUserRef": "your-internal-user-id-12345",
  "requestedClaims": ["age_over_18", "citizenship"],
  "returnUrl": "https://yourapp.com/onboarding/done"
}
```

Response (200 OK)

```
{
  "success": true,
  "data": {
    "requestId": "pvr_1744531200000_9ezjx8999",
    "status": "pending_kyc",
    "flow": "web",
    "partnerUserRef": "your-internal-user-id-12345",
    "requestedClaims": ["age_over_18", "citizenship"],
    "expiresAt": "2026-04-14T10:00:00.000Z",
    "createdAt": "2026-04-13T10:00:00.000Z",
    "token": "eyJhbGciOi...",
    "redirectUrl": "https://{web_url}/verify?requestId=pvr_...&token=eyJhbGciOi..."
  }
}
```

Additional Response Fields (Option 2)

Field	Type	Description
flow	string	Always <code>web</code> for the <code>partnerUserRef</code> path
partnerUserRef	string	Echoes the reference you sent
token	string	Short-lived token embedded in the redirect URL
redirectUrl	string	QIE-hosted onboarding page. Redirect the user here (see Step 2)

A brand-new user always returns `status: pending_kyc` — there is no prior KYC to reuse.

Step 2 — Handle User KYC Status

The response from Step 1 tells you whether the user has already completed KYC through QIE. There are two cases.

Case A — User Already Verified

The response contains `status: "pending_consent"` with no `redirectUrl`. The user receives a push notification in the QIE Wallet and will approve or reject your data request there. No additional action is required on your side — proceed to Step 3.

How to simulate in sandbox

Complete KYC on the QIE Wallet first (on test network), then create a verification request from your platform.

Case B — User Not Yet Verified

The response contains `status: "pending_kyc"` and a `redirectUrl`. The user must complete KYC before they can approve your request. Redirect them to the provided URL:

Path	Where <code>redirectUrl</code> points
Option 1 (identifier)	Instructions page guiding the user to complete KYC inside the QIE Wallet, then approve in-wallet.
Option 2 (partnerUserRef)	QIE-hosted onboarding page (carries the <code>token</code>) where the user creates a wallet, completes KYC, and approves — then returns to your <code>returnUrl</code> .

Redirect user to KYC flow

```
if (response.data.status === 'pending_kyc') {
  window.location.href = response.data.redirectUrl;
}
```

Once the user completes KYC and responds to your request, the updated status will be available via polling in Step 3.

How to simulate in sandbox

Create a verification request using an identifier that has not completed KYC. Follow the push notification in the QIE Wallet, complete KYC, then approve or reject the request. For Option 2, create the request with a `partnerUserRef` and follow the hosted onboarding page.

Step 3 — Poll for Consent

GET /partners/verification-requests/:requestId

Retrieves the current status of a verification request

Poll this endpoint while the status is `pending_kyc` or `pending_consent`. Stop as soon as you receive a terminal status:

Status	Meaning	Required Action
<code>consent_given</code>	User approved the request	Proceed to Step 4 to claim verified data
<code>consent_rejected</code>	User rejected the request	Do not retry immediately — prompt the user to restart if needed
<code>expired</code>	Request timed out before the user acted	Prompt the user to initiate a new request
<code>failed</code>	User's KYC was rejected by QIE	Inform the user — no further action possible on this request

Polling Recommendation

Poll at a reasonable interval (e.g. every 5–10 seconds) rather than continuously.

Always stop polling on any terminal status: `consent_given`, `consent_rejected`, `expired`, or `failed`.

Step 4 — Claim Verified Data

POST /vc/partner/claim-and-verify/

Claims the cryptographically-signed verified credential for an approved request

Time-Sensitive

Claim as soon as status === "consent_given". You have a 24-hour window, after this the user will need to approve the request again.

Request Body

POST /vc/partner/claim-and-verify/

```
{
  "requestId": "pvr_1744531200000_9ezjx8999"
}
```

Response (200 OK)**Response – Verifiable Credential**

```

{
  "success": true,
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://w3id.org/security/suites/ecdsa-2019/v1",
    "https://did-stapi.qie.digital/contexts/v1.json"
  ],
  "credentialId": "vc:qie:a1b2c3d4e5f6...",
  "type": ["VerifiableCredential", "KYCCredential"],
  "subject": "did:qie:a1b2c3d4e5f6...",
  "issuer": {
    "id": "did:qie:0xQIEPlatformWalletAddress",
    "name": "QIE Identity Platform"
  },
  "issuanceDate": "2026-01-15T08:00:00.000Z",
  "expirationDate": "2028-01-15T08:00:00.000Z",
  "publicClaims": {
    "kyc_verified": true,
    "liveness_completed": true,
    "documents_verified": true,
    "verification_level": "basic"
  },
  "requestedClaims": {
    "age_over_18": true,
    "citizenship": "IN"
  },
  "proof": {
    "type": "EcdsaSecp256k1Signature2019",
    "created": "2026-04-13T10:12:00.000Z",
    "proofPurpose": "assertionMethod",
    "verificationMethod": "did:qie:platform#keys-1",
    "publicKey": "0xQIEPlatformWalletAddress",
    "proofValue": "0x..."
  },
  "verification": {
    "signatureValid": true,
    "commitmentValid": true,
    "notExpired": true,
    "notRevoked": true,
    "verifiedAt": "2026-04-13T10:15:00.000Z",
    "method": "single-step-verification"
  }
}

```

Response Object Reference

Field	Type	Description
credentialId	string	Unique identifier for the issued Verifiable Credential
subject	string	DID (Decentralised Identifier) of the verified user
issuer	object	QIE Identity Platform details
issuanceDate	string	When the credential was issued (ISO 8601)
expirationDate	string	When the credential expires (ISO 8601)
publicClaims	object	Standard KYC verification flags (kyc_verified, liveness_completed, etc.)
requestedClaims	object	The specific claims you requested — you receive only these, nothing extra
proof	object	Cryptographic proof — use to verify the credential was issued by QIE
verification	object	Real-time verification result (signature, expiry, revocation status)

Data Minimisation

You receive only the claims you configured in requestedClaims — no raw documents, no additional data. Use the proof object to cryptographically verify the credential was genuinely issued by QIE.

Error Reference

HTTP Code	Meaning	Resolution
400	Bad request	Check your request body — a required field may be missing or malformed (e.g. both or neither of <code>identifier</code> / <code>partnerUserRef</code>)
401	Authentication failed	Verify your signature formula and ensure the timestamp is current
409	Conflict	A verified record already exists for this user with your organisation
500	Server error	Retry after a short delay — if persistent, contact QIE support

Sandbox Environment

The sandbox environment is available for integration testing. All endpoints are accessible at the base URL using sandbox credentials (`pk_test_...`).

Sandbox Notice

No real OCR or document data extraction takes place in sandbox. Returned claim values may not match actual user input.

Use the sandbox to test your integration flow and error handling only. Never use sandbox for validating real user data.

Best Practices

Always offer Option 1 alongside Option 2

The `identifier` flow is required for returning QIE Wallet users; the `partnerUserRef` web flow is only for users with no wallet yet. A web-only integration breaks identity reuse and triggers duplicate-KYC rejection.

Send exactly one identity field

Provide `identifier` or `partnerUserRef` — never both, never neither.

Keep `partnerUserRef` stable

Use the same reference for the same user — it is used for correlation and deduplication.

Request only the claims you need

Users see every field you request. Fewer claims = higher approval rates.

Regenerate signatures on every request

Each API call requires a fresh timestamp and signature. Never reuse them.

Claim within 24 hours

Once a user approves (`consent_given`), you have 24 hours to claim. After that, they must approve again.

Handle `consent_rejected` gracefully

Do not immediately retry after rejection — repeated requests create a poor user experience.

Test thoroughly in sandbox first

Validate your full integration flow before requesting production credentials.

Poll at reasonable intervals

5–10 second intervals are sufficient. Stop polling immediately on any terminal status.